Gerência de configuração

Utilizando o terraform e o ansible

Prof. Diego Cardoso Borda Castro

Introdução

- Terraform
  - Provisionamento de infra
  - Está relacionado a criação dos recursos
- Ansible
  - Gerenciamento de configuração
  - Está relacionado a configuração de recursos já provisionados

Não vão competir e sim se complementar

Introdução

Tarefa	Ansible	Terraform
Instalar software	✅	❌
Configurar servidor	✅	❌
Criar VM	⚠️	✅
Alterar tamanho da VM	⚠️	✅
Criar VNet	❌	✅
Criar NSG	❌	✅
Gerenciar infraestrutura Azure	⚠️	✅

Introdução

- Terraform
  - Provisionamento de infra
  - Está relacionado a criação dos recursos
- Ansible
  - Gerenciamento de configuração
  - Está relacionado a configuração de recursos já provisionados

Introdução

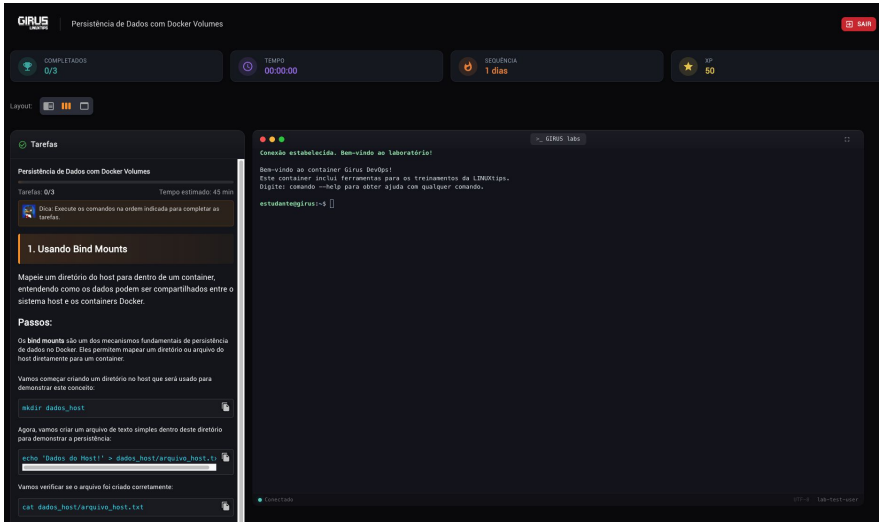
- Aprender Terraform e Ansible normalmente exige acesso a provedores de nuvem como:
  - AWS
  - Azure
  - Google Cloud
- Isso pode gerar
  - Custos inesperados
  - Necessidade de cartão de crédito



Ambientes simulados

Simuladores de devops

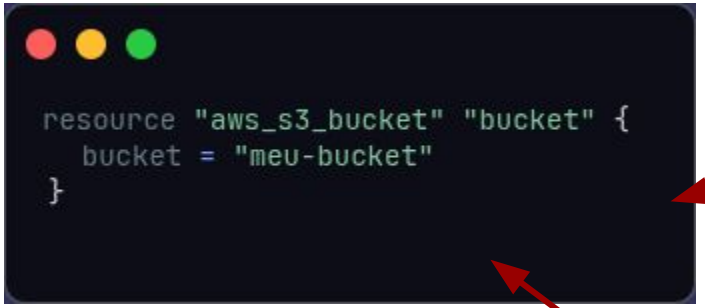
- Girus
  - Ferramenta criada pela LinuxTips
    - Facilitar o aprendizado
    - Evitar custos em provedores reais
    - Ferramenta
      - Terraform
      - Ansible
      - Docker
      - Kubernetes
      - …



Simuladores de devops

- LocalStack
  - Plataforma que simula diversos serviços da AWS localmente
  - Em vez de criar recursos na AWS real
- Existem outros
  - Azurite
  - Floci-az
  - Localgcp
  - …

Simuladores de devops



- Na AWS
  - Cria um bucket real

- No LocalStack:
  - Cria um bucket simulado localmente

Localstack

- Criação da imagem do localstack
- A versão free não sobe container
  - Criamos uma vm pra poder testar o ansible

```yaml
services:
  localstack:
    image: localstack/localstack:3.8
    container_name: localstack
    ports:
      - "4566:4566"
    environment:
      - SERVICES=ec2
      - DEFAULT_REGION=us-east-1
      - AWS_DEFAULT_REGION=us-east-1
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
    healthcheck:
      test: ["CMD", "bash", "-c", "curl -sf http://localho
      interval: 5s
      timeout: 10s
      retries: 20
      start_period: 10s

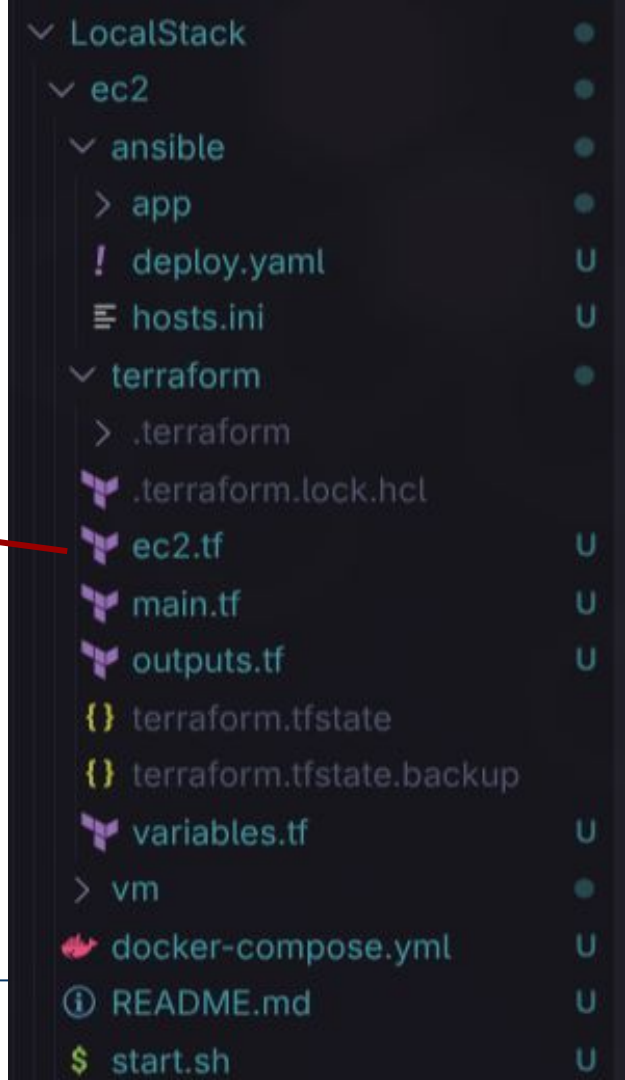
  vm:
    build: ./vm
    container_name: vm
    ports:
      - "2222:22"
      - "5000:5000"
    healthcheck:
      test: ["CMD-SHELL", "bash -c 'echo > /dev/tcp/localh
      interval: 5s
      timeout: 5s
      retries: 10
      start_period: 5s
```

Localstack

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region                      = "us-east-1"
  access_key                  = "test"
  secret_key                  = "test"
  skip_credentials_validation = true
  skip_metadata_api_check     = true
  skip_requesting_account_id  = true

  endpoints {
    ec2 = "http://localhost:4566"
  }
}
```

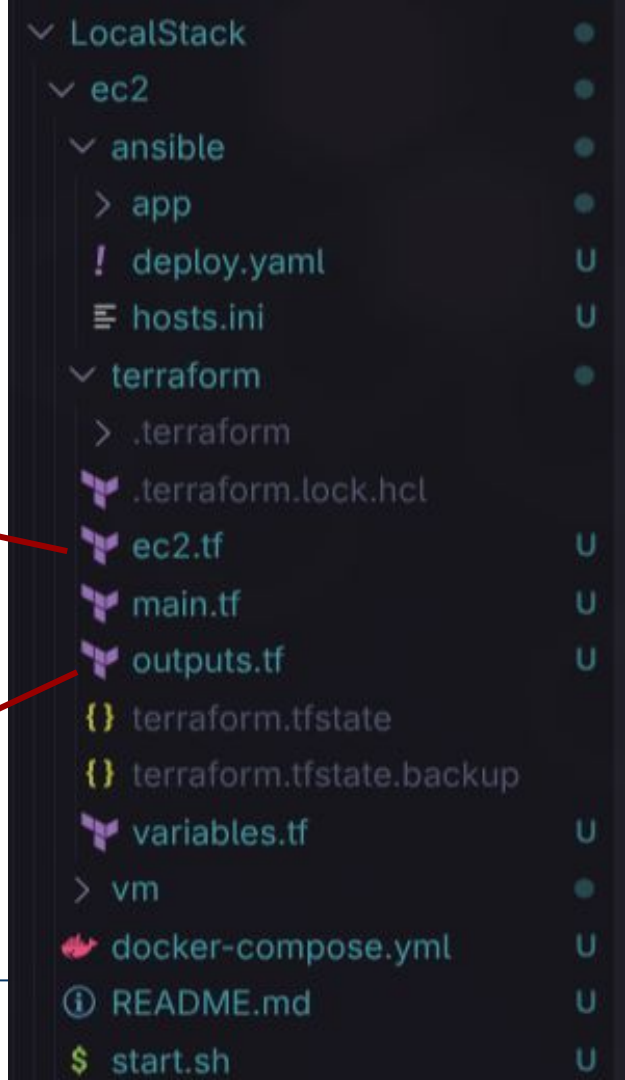


Localstack

```
resource "aws_instance" "api_server" {
  ami           = "ami-ff0fea8310f3"
  instance_type = var.instance_type

  tags = {
    Name = var.server_name
  }
}
```

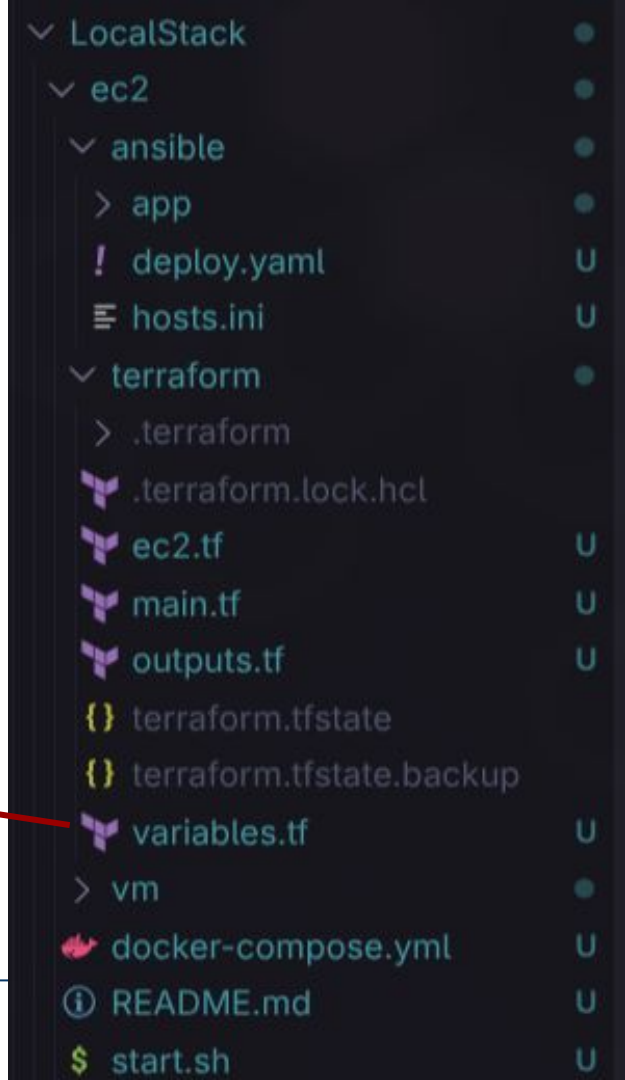
```
output "instance_id" {
  value = aws_instance.api_server.id
}
```



Localstack

```
variable "instance_type" {
  description = "Tipo da instância EC2"
  type        = string
  default     = "t2.micro"
}

variable "server_name" {
  description = "Nome do servidor"
  type        = string
  default     = "api-server"
}
```



```yaml
- name: Deployar API na EC2
  hosts: api_server
  become: true

  tasks:

    - name: Criar diretorio da aplicacao
      file:
        path: /opt/api
        state: directory

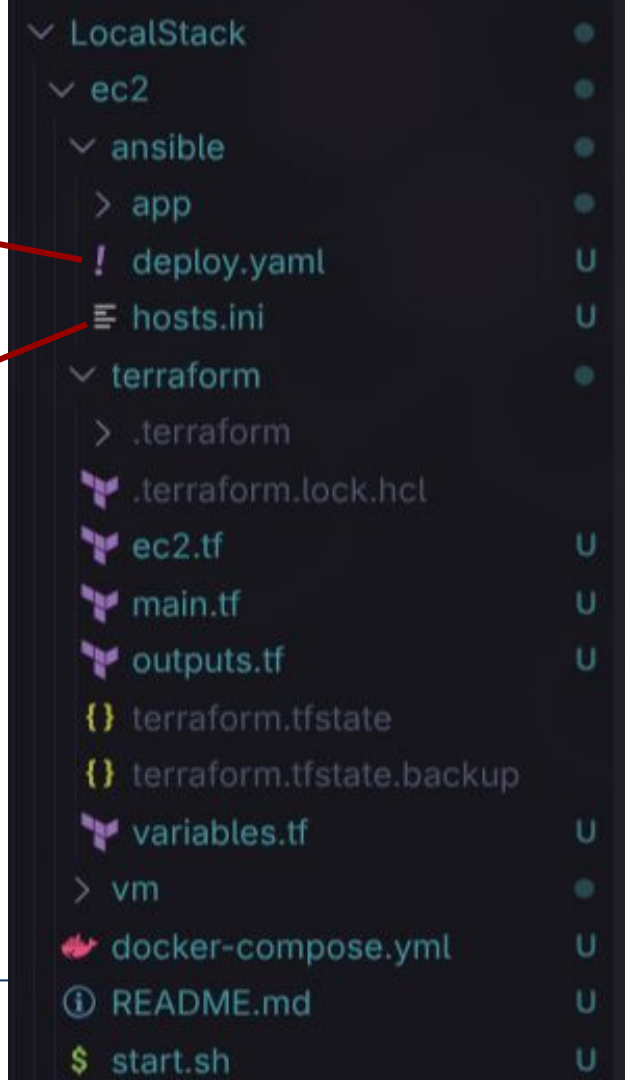
    - name: Copiar arquivos da API
      copy:
        src: app/
        dest: /opt/api/

    - name: Instalar dependencias npm
      shell: npm install
      args:
        chdir: /opt/api

    - name: Iniciar API em background
      shell: node api.js >> /var/log/api.log 2>&1
      args:
        chdir: /opt/api
      async: 1000
      poll: 0
```

```
[api_server]
localhost

[api_server:vars]
ansible_port=2222
ansible_user=root
ansible_password=root
```

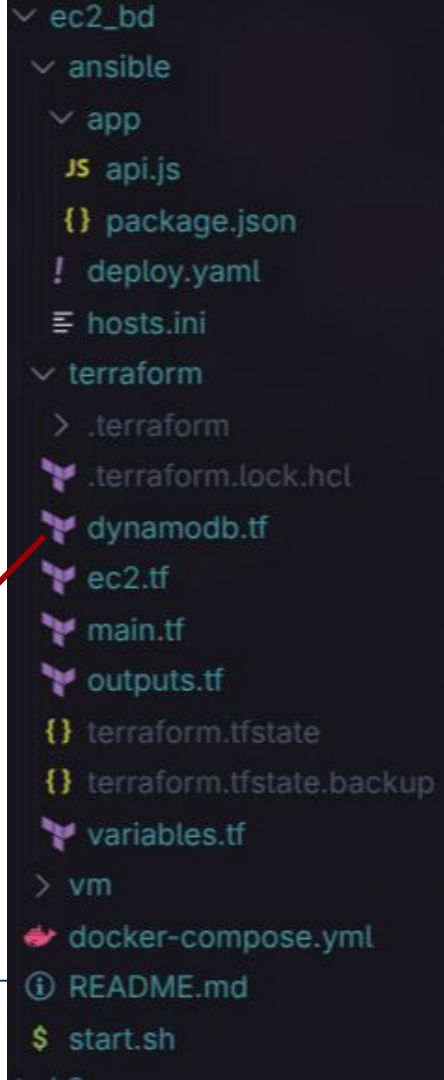


Localstack

- Terraform
  - Adicionando um dynamodb
- Ansible
  - Migrations
  - Seeds

```
resource "aws_dynamodb_table" "alunos" {
  name         = "alunos"
  billing_mode = "PAY_PER_REQUEST"
  hash_key     = "id"

  attribute {
    name = "id"
    type = "S"
  }
}
```

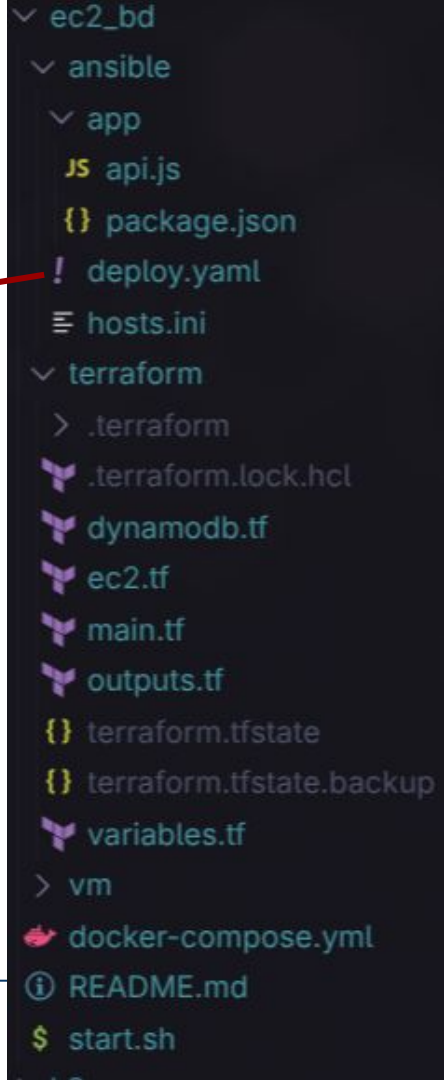


Localstack

```
- name: Inserir alunos no DynamoDB
  shell: |
    AWS_ACCESS_KEY_ID=test AWS_SECRET_ACCESS_KEY=test \
    aws --endpoint-url={{ localstack_endpoint }} --region us-east-1 \
    dynamodb put-item --table-name alunos --item '{{ item }}'
  loop:
    - '{"id":{"S":"1"},"nome":{"S":"João Silva"},"curso":{"S":"DevOps"},
    - '{"id":{"S":"2"},"nome":{"S":"Maria Santos"},"curso":{"S":"Kuberne
    - '{"id":{"S":"3"},"nome":{"S":"Pedro Costa"},"curso":{"S":"Terrafor
```

```
aws_dynamodb_table.alunos: Creation complete after 0s [id=alunos]
aws_instance.api_server: Still creating... [00m10s elapsed]
aws_instance.api_server: Creation complete after 10s [id=i-e46a049a9042a28ea]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
```



Floci

- Diferente do LocalStack, pode ser usado para várias clouds
- Não precisa criar a vm
  - Já retorna um container

```yaml
services:
  floci:
    image: floci/floci:latest
    container_name: floci
    ports:
      - "4566:4566"
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
    healthcheck:
      test: ["CMD-SHELL", "curl -sf http://localhost:4
      interval: 5s
      timeout: 10s
      retries: 20
      start_period: 15s
```

Floci

- Como não usamos o container do docker
  - Temos que pegar o output do terraform para acessar a vm e subir a api

```
#!/bin/bash
set -e

docker compose up -d --wait

cd terraform
terraform init -input=false -upgrade
terraform apply -auto-approve

EC2_IP=$(terraform output -raw instance_public_ip)
SSH_KEY=$(terraform output -raw private_key_path)
cd ..

until ssh -o StrictHostKeyChecking=no -o ConnectTimeout=
  sleep 10
done

cat > ansible/hosts.ini <<EOF
[api_server]
${EC2_IP}

[api_server:vars]
ansible_user=ubuntu
ansible_ssh_private_key_file=${SSH_KEY}
ansible_ssh_extra_args=-o StrictHostKeyChecking=no -o Us
EOF

ansible-playbook -i ansible/hosts.ini ansible/deploy.yam

curl -s "http://$EC2_IP:5000/health"
curl -s "http://$EC2_IP:5000/"
```

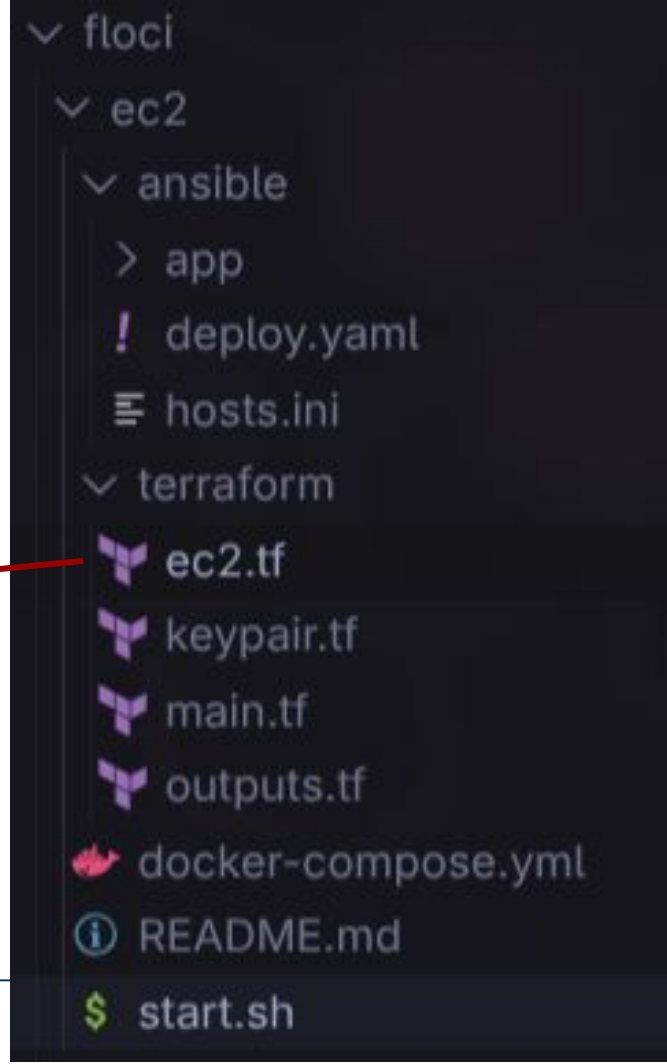
Floci

```hcl
data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"]

  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-*"]
  }
}

resource "aws_instance" "api_server" {
  ami           = data.aws_ami.ubuntu.id
  instance_type = "t3.micro"
  key_name      = aws_key_pair.app_key.key_name

  tags = {
    Name = "hello-world-api"
  }
}
```

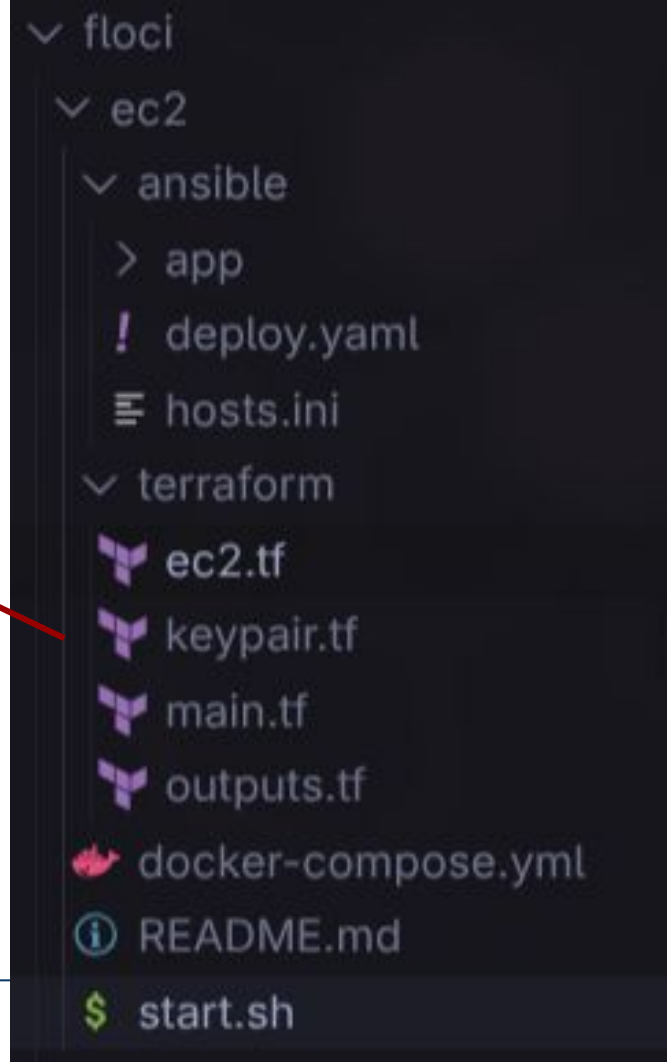


Floci

```
resource "tls_private_key" "ssh_key" {
  algorithm = "RSA"
  rsa_bits  = 4096
}

resource "aws_key_pair" "app_key" {
  key_name   = "floci-api-key"
  public_key = tls_private_key.ssh_key.public_key_openssh
}

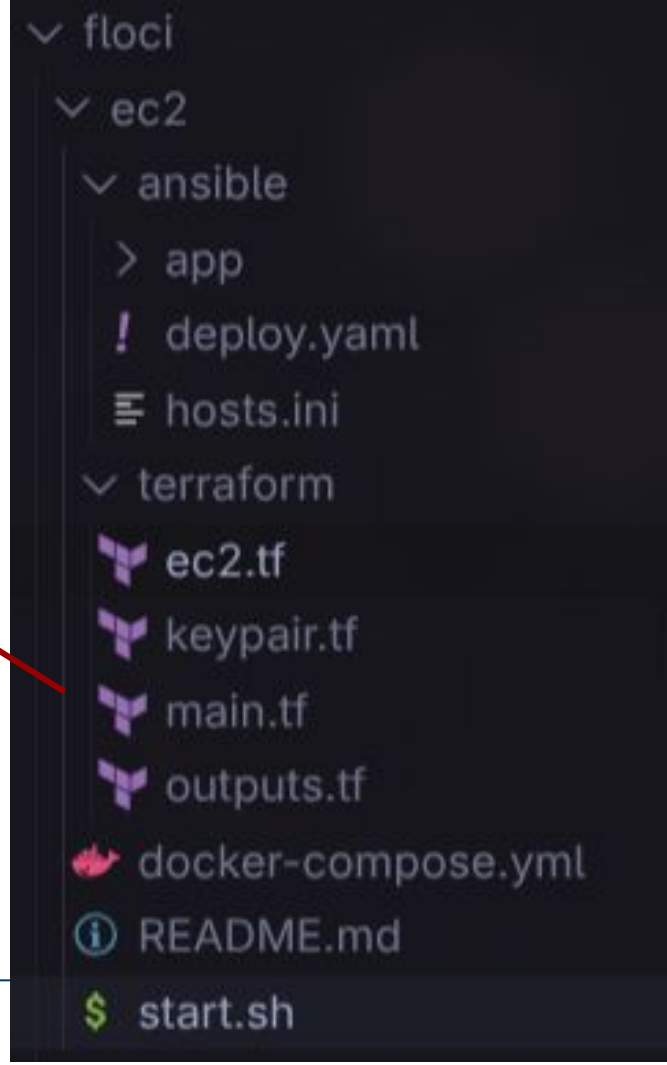
resource "local_sensitive_file" "private_key" {
  content         = tls_private_key.ssh_key.private_key_pem
  filename        = "${path.module}/../floci-api.pem"
  file_permission = "0600"
}
```



```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
    tls = {
      source  = "hashicorp/tls"
      version = "~> 4.0"
    }
    local = {
      source  = "hashicorp/local"
      version = "~> 2.0"
    }
  }
}

provider "aws" {
  region                      = "us-east-1"
  access_key                  = "test"
  secret_key                  = "test"
  skip_credentials_validation = true
  skip_metadata_api_check     = true
  skip_requesting_account_id  = true

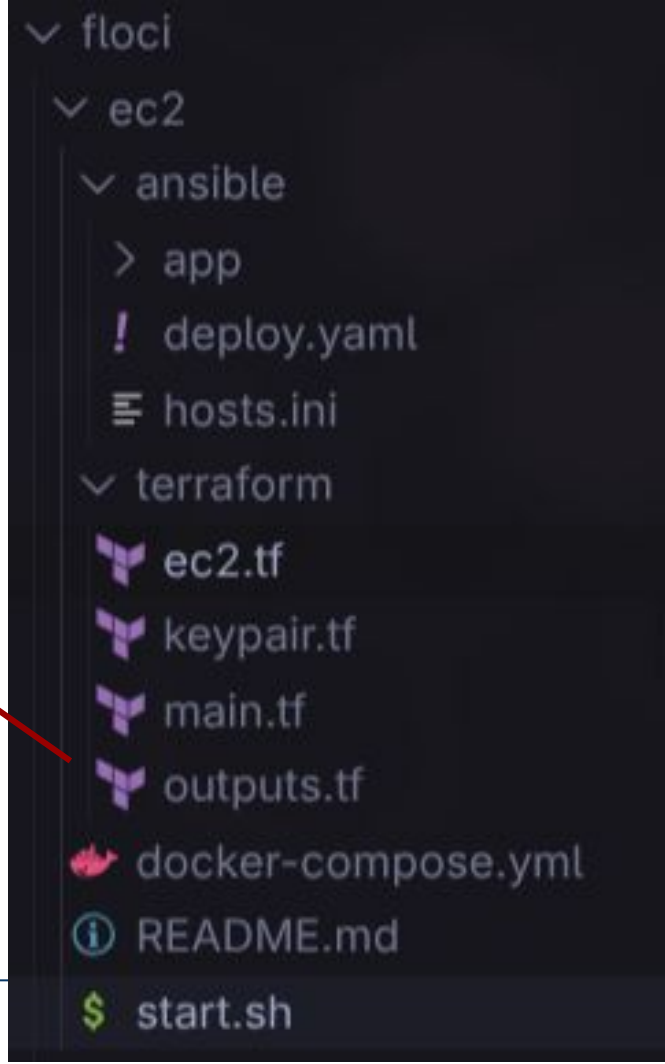
  endpoints {
    ec2 = "http://localhost:4566"
  }
}
```



Floci

```
output "instance_public_ip" {
  description = "IP da instancia EC2 criada pelo Floci"
  value       = aws_instance.api_server.public_ip
}

output "private_key_path" {
  description = "Caminho da chave SSH gerada"
  value       = local_sensitive_file.private_key.filename
}
```



```yaml
- name: Deployar API Hello World
  hosts: api_server
  become: true

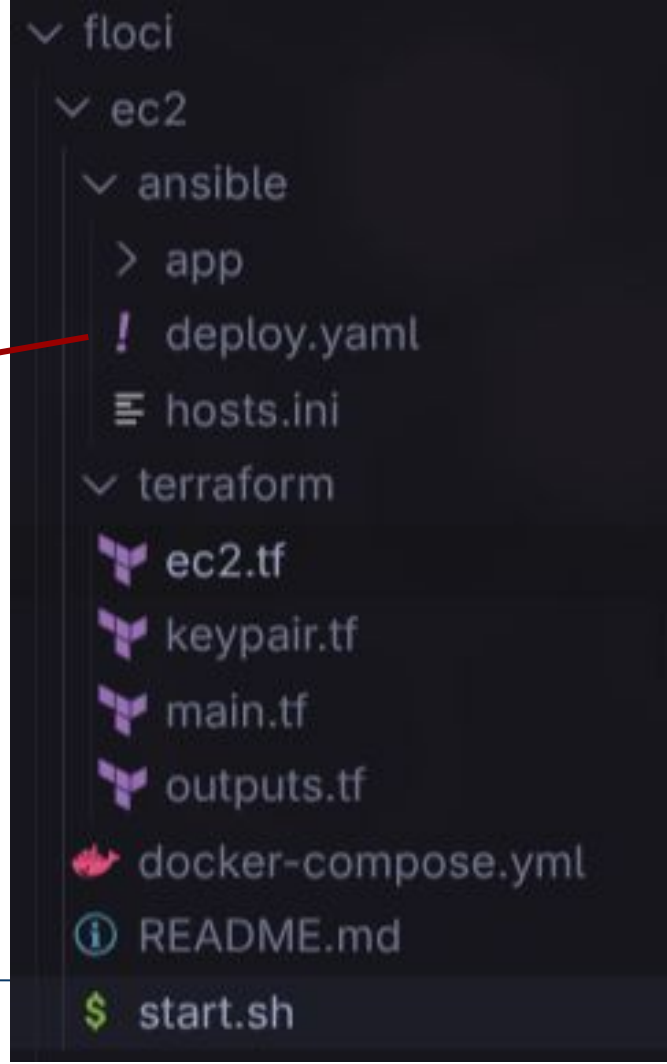
  tasks:
    - name: Atualizar apt cache
      apt:
        update_cache: yes
        cache_valid_time: 3600

    - name: Instalar curl
      apt:
        name: curl
        state: present

    - name: Adicionar repositorio Node.js 20
      shell: curl -fsSL https://deb.nodesource.com/setup_20.x | bash -
      args:
        creates: /etc/apt/sources.list.d/nodesource.list

    - name: Instalar Node.js
      apt:
        name: nodejs
        state: present
        update_cache: yes

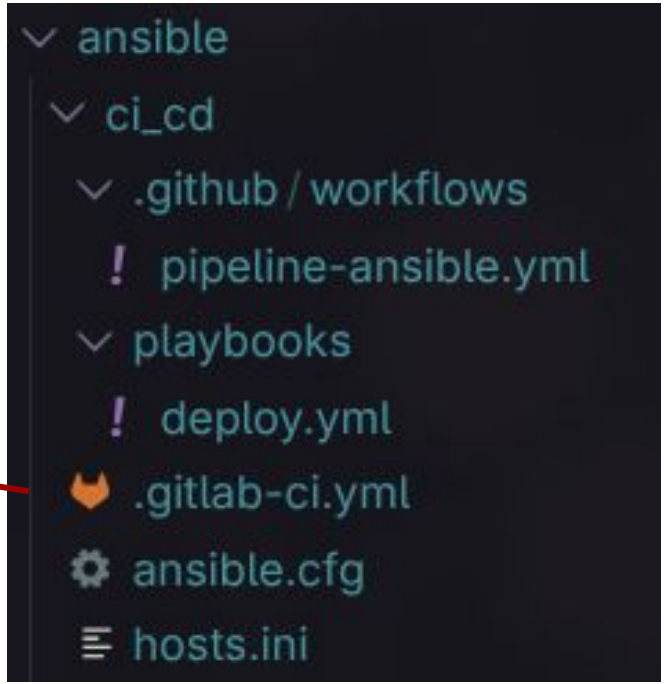
    - name: Criar diretorio da aplicacao
      file:
        path: /opt/api
        state: directory
        mode: '0755'
```



CI / CD com Ansible

```yaml
stages:
  - deploy

deploy:
  image: python:3.11-slim
  stage: deploy
  before_script:
    - pip install ansible --quiet
  script:
    - ansible-playbook ansible/ci_cd/playbooks/deploy.yml
  variables:
    DEPLOY_HOOK_URL: $RENDER_DEPLOY_HOOK
    APP_DIR: ./segunda-pipeline
  only:
    - main
```



```yaml
- name: Deploy API JS via Deploy Hook
  hosts: localhost
  connection: local

  vars:
    deploy_hook_url: "{{ lookup('env', 'DEPLOY_HOOK_URL') }}"
    app_dir: "{{ lookup('env', 'APP_DIR') | default('.') }}"

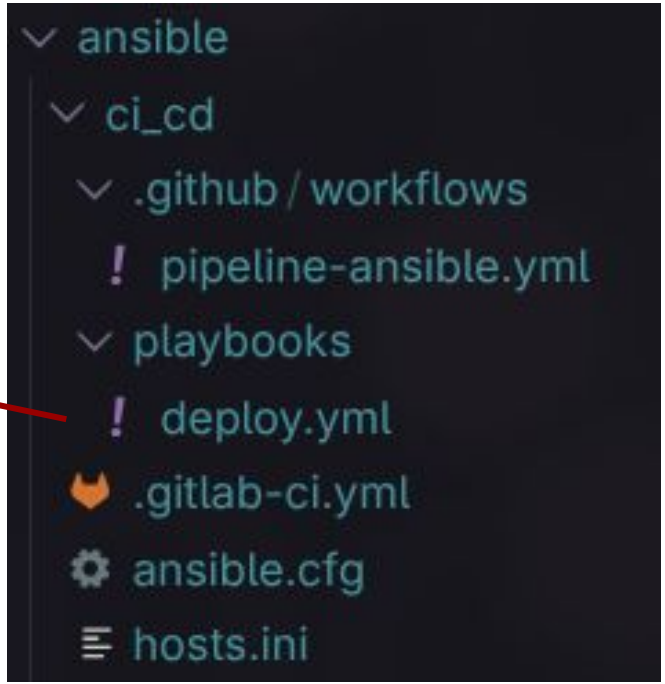
  tasks:

    - name: Instalar dependências
      command: npm install
      args:
        chdir: "{{ app_dir }}"

    - name: Build da aplicação
      command: npm run build
      args:
        chdir: "{{ app_dir }}"

    - name: Disparar Deploy Hook
      uri:
        url: "{{ deploy_hook_url }}"
        method: POST
        status_code: [200, 201, 204]
        timeout: 30
      register: deploy_response

    - name: Confirmar Deploy
      debug:
        msg: "Deploy disparado com sucesso. HTTP: {{ deploy_response.status }}"
```



```yaml
name: Pipeline com Ansible
run-name: Pipeline Ansible executada por ${{ github.actor }}

on:
  push:
    branches: [main]
  workflow_dispatch:

jobs:

  deploy:
    name: Deploy
    runs-on: ubuntu-latest

    steps:
      - name: Checkout
        uses: actions/checkout@v5

      - name: Instalar Ansible
        run: pip install ansible

      - name: Deploy via Ansible
        run: ansible-playbook ansible/ci_cd/playbooks/deploy.yml
        env:
          DEPLOY_HOOK_URL: ${{ secrets.RENDER_DEPLOY_HOOK }}
          APP_DIR: ./segunda-pipeline
```

